MINISTRY OF EDUCATION AND RESEARCH OF THE REPUBLIC OF MOLDOVA
Technical University of Moldova
Faculty of Computers Informatics and Microelectronics
Department of Software Engineering and Automation

Student:
Tacu Igor
FAF-242

REPORT

Laboratory Works No 7 and No 8

Three Small Programs Processes Redirection and a Pipe

The Same Calls Inside Your Own OS

Operating Systems

Checked by:
Universitary Assistant
Reițman Patricia

UTM
Chișinău 2026

Purpose of the Laboratory Works

Laboratory Work No. 7 investigates process creation, program replacement, process waiting, zombie states, file-descriptor redirection, and pipelines on Linux. Laboratory Work No. 8 repeats the process-control operations inside xv6 and implements bidirectional process communication with two pipes. Together, the works connect familiar shell behavior to the system calls and descriptor rules that implement it.

Objectives

1. Create a child with fork, replace it with exec, and collect its termination status with wait or waitpid.

2. Observe a zombie process and explain why waiting removes it from the process table.

3. Redirect standard output by closing descriptor 1 and reopening a file in its place.

4. Construct ls /etc | wc -l with a pipe and two children, then explain an intentional EOF deadlock.

5. Register and execute new user programs inside xv6.

6. Exchange a byte between parent and child with separate parent-to-child and child-to-parent pipes.

Experimental Environment

The Linux experiments were performed on a remote Ubuntu 24.04.3 LTS system with GCC 13.3.0. The Linux programs were executed from ~/os-lab7. The xv6-riscv tree was compiled with the RISC-V GNU toolchain and executed under qemu-system-riscv64. Lab 7 commands were run in the remote root session, while the xv6 work was performed as user loredana.

Report Structure

Part I presents the four Linux experiments from Laboratory Work No. 7. Part II presents the xv6 run and pingpong programs from Laboratory Work No. 8. The appendices contain the complete source code, followed by a joint conclusion and critical analysis.

1 Laboratory Work No 7 Three Small Programs

1.1 Process Creation Program Replacement and Waiting

The program first printed its parent PID, then called fork. The child printed its own PID and the PID returned by getppid before executing ls -l. The parent blocked in waitpid until the child finished. The same child path was then tested with date and with nosuchprog.

```
gcc -Wall -Wextra -o run run.c
./run
```

```
    waitpid(pid, &status, 0);
    printf("parent: child %d ended with status %d\n",
           pid, WEXITSTATUS(status));
    return 0;
}
EOF

gcc -Wall -Wextra -o run run.c
./run
parent: my PID is 1787246
child:  my PID is 1787247, my parent is 1787246
total 24
-rwxr-xr-x 1 root root 16400 Oct 19 12:27 run
-rw-r--r-- 1 root root   710 Oct 19 12:27 run.c
parent: child 1787247 ended with status 0
root@ubuntu-4gb-hel1-1:~/os-lab7# cp run.c run-ls.c
root@ubuntu-4gb-hel1-1:~/os-lab7# cp run.c zombie.c
root@ubuntu-4gb-hel1-1:~/os-lab7# sed -i '/execlp(/c\        execlp("date", "date", (char *)NULL);' run.c
root@ubuntu-4gb-hel1-1:~/os-lab7# gcc -Wall -Wextra -o run run.c
root@ubuntu-4gb-hel1-1:~/os-lab7# ./run
parent: my PID is 1787897
child:  my PID is 1787898, my parent is 1787897
Mon Oct 19 12:28:17 UTC 2026
parent: child 1787898 ended with status 0
root@ubuntu-4gb-hel1-1:~/os-lab7# sed -i '/execlp(/c\        execlp("nosuchprog", "nosuchprog", (char *)NULL);' run.c
root@ubuntu-4gb-hel1-1:~/os-lab7# gcc -Wall -Wextra -o run run.c
root@ubuntu-4gb-hel1-1:~/os-lab7# ./run
parent: my PID is 1788740
child:  my PID is 1788741, my parent is 1788740
exec: No such file or directory
parent: child 1788741 ended with status 127
root@ubuntu-4gb-hel1-1:~/os-lab7# cp run-ls.c run.c
root@ubuntu-4gb-hel1-1:~/os-lab7# gcc -Wall -Wextra -o run run.c
root@ubuntu-4gb-hel1-1:~/os-lab7# sed -i '/waitpid(pid, &status, 0);/i\    sleep(20);' zombie.c
```

Figure 1 - Linux fork exec and wait results

Observation

perror("exec") was printed only for nosuchprog because a successful exec replaces the child image and never returns. The child reported the original process PID as its parent. Both ls and date ended with status 0, while the missing program ended with status 127 as selected by exit(127).

1.2 A Zombie Process

A 20-second sleep was inserted before waitpid. During that interval, ls had already terminated, but its parent had not yet collected the status. The process table therefore displayed PID 1789518 with state Z and the command [ls] <defunct>.

```
gcc -Wall -Wextra -o zombie zombie.c
./zombie > /dev/null &
sleep 2
ps -o pid,ppid,stat,cmd
sleep 20
ps -o pid,ppid,stat,cmd
```

```
child:  my PID is 1788741, my parent is 1788740
exec: No such file or directory
parent: child 1788741 ended with status 127
root@ubuntu-4gb-hel1-1:~/os-lab7# cp run-ls.c run.c
root@ubuntu-4gb-hel1-1:~/os-lab7# gcc -Wall -Wextra -o run run.c
root@ubuntu-4gb-hel1-1:~/os-lab7# sed -i '/waitpid(pid, &status, 0);/i\    sleep(20);' zombie.c
root@ubuntu-4gb-hel1-1:~/os-lab7# gcc -Wall -Wextra -o zombie zombie.c
root@ubuntu-4gb-hel1-1:~/os-lab7# ./zombie > /dev/null &
[1] 1789517
root@ubuntu-4gb-hel1-1:~/os-lab7# sleep 2
root@ubuntu-4gb-hel1-1:~/os-lab7# ps -o pid,ppid,stat,cmd
    PID    PPID STAT CMD
1784391 1784256 Ss   -bash
1789517 1784391 S    ./zombie
1789518 1789517 Z    [ls] <defunct>
1789748 1784391 R+   ps -o pid,ppid,stat,cmd
root@ubuntu-4gb-hel1-1:~/os-lab7# sleep 20
[1]+  Done                    ./zombie > /dev/null
```

```
root@ubuntu-4gb-hel1-1:~/os-lab7# ps -o pid,ppid,stat,cmd
    PID    PPID STAT CMD
1784391 1784256 Ss   -bash
1790485 1784391 R+   ps -o pid,ppid,stat,cmd
root@ubuntu-4gb-hel1-1:~/os-lab7# cat > redirect.c <<'EOF'
```

Figure 2 - Zombie state before and after waitpid

Observation

Z denotes a terminated child whose exit status has not yet been reaped. After the delay, waitpid(pid, &status, 0) collected the child and the defunct entry disappeared.

1.3 Standard Output Redirection

The child closed descriptor 1 and opened out.txt. Since open returns the smallest available descriptor, it returned 1. The unchanged ls process therefore wrote its standard output to the file, while the diagnostic sent through stderr remained visible on the terminal.

```
gcc -Wall -Wextra -o redirect redirect.c
./redirect
head -3 out.txt
```

```
    }

    waitpid(pid, NULL, 0);
    printf("done: the listing is in out.txt\n");
    return 0;
}
EOF

gcc -Wall -Wextra -o redirect redirect.c
./redirect
head -3 out.txt
open gave descriptor 1
done: the listing is in out.txt
total 72
-rw-r--r-- 1 root root     0 Oct 19 12:30 out.txt
-rwxr-xr-x 1 root root 16344 Oct 19 12:30 redirect
```

Figure 3 - Redirecting ls output through descriptor 1

Observation

open returned descriptor 1 because it was the lowest free descriptor. The listing did not appear on screen because ls continued writing to descriptor 1, which now referred to out.txt. Standard error would be redirected by closing descriptor 2 before open.

1.4 Constructing a Pipeline

One child connected its standard output to the pipe's write end, while the second connected its standard input to the read end. The parent closed both pipe descriptors and waited for the two children. Both the program and the shell pipeline counted 214 entries in /etc.

```
gcc -Wall -Wextra -o pipeline pipeline.c
./pipeline
ls /etc | wc -l
```

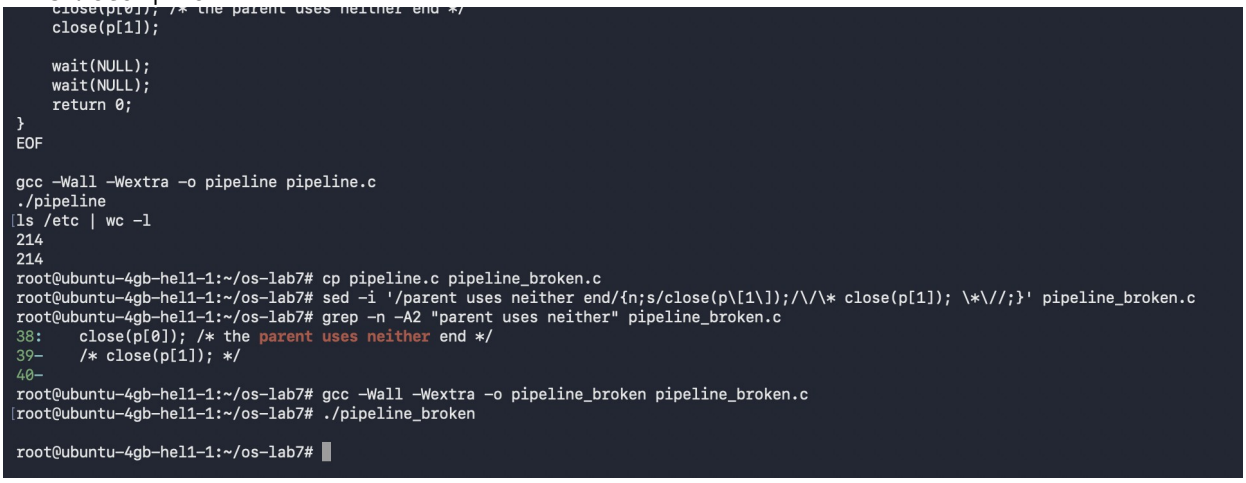


Figure 4 - Correct pipeline and intentional open-writer failure

Observation

The two correct commands produced the same result, 214. After the parent's close(p[1]) was commented out, ls could terminate but wc remained blocked. wc was waiting for EOF, which could not arrive while the parent still held an open write descriptor.

2 Laboratory Work No 8 The Same Calls Inside xv6

2.1 Registering User Programs

The new source files user/run.c and user/pingpong.c were added to the UPROGS list as $U/_run and $U/_pingpong. The xv6 build then linked the programs and copied them into fs.img.

```
loredana@ubuntu-4gb-hel1-1:~/xv6-riscv$ grep -nE 'UPROGS|_run|_pingpong' Makefile
132:UPROGS=\
133:    $U/_run\
134:    $U/_pingpong\
160:fs.img: mkfs/mkfs README $(UPROGS)
161:    mkfs/mkfs fs.img README $(UPROGS)
171:    $(UPROGS)
loredana@ubuntu-4gb-hel1-1:~/xv6-riscv$
```

Figure 5 - xv6 UPROGS registration and source-file verification

2.2 fork exec and wait Inside xv6

The xv6 program used the same three-stage pattern as Linux. The parent called fork, the child called exec, and the parent called wait. In the first test, exec replaced the child with echo, which printed the requested message and exited successfully.

```
xv6 kernel is booting

hart 2 starting
hart 1 starting
init: starting sh
$ run
parent: my pid is 3
child: my pid is 4
hello from the child
parent: child 4 ended with status 0
$
```

Figure 6 - xv6 run executing echo

Observation

The Linux calls were fork, execlp, and waitpid; the xv6 calls were fork, exec, and wait. Their roles are equivalent. The text 'hello from the child' was printed by echo after exec replaced the child's copy of run.

2.3 Replacing the Child with ls

The argument vector and exec target were changed from echo to ls. The output listed the xv6 file system, including run and pingpong, and the parent received status 0.

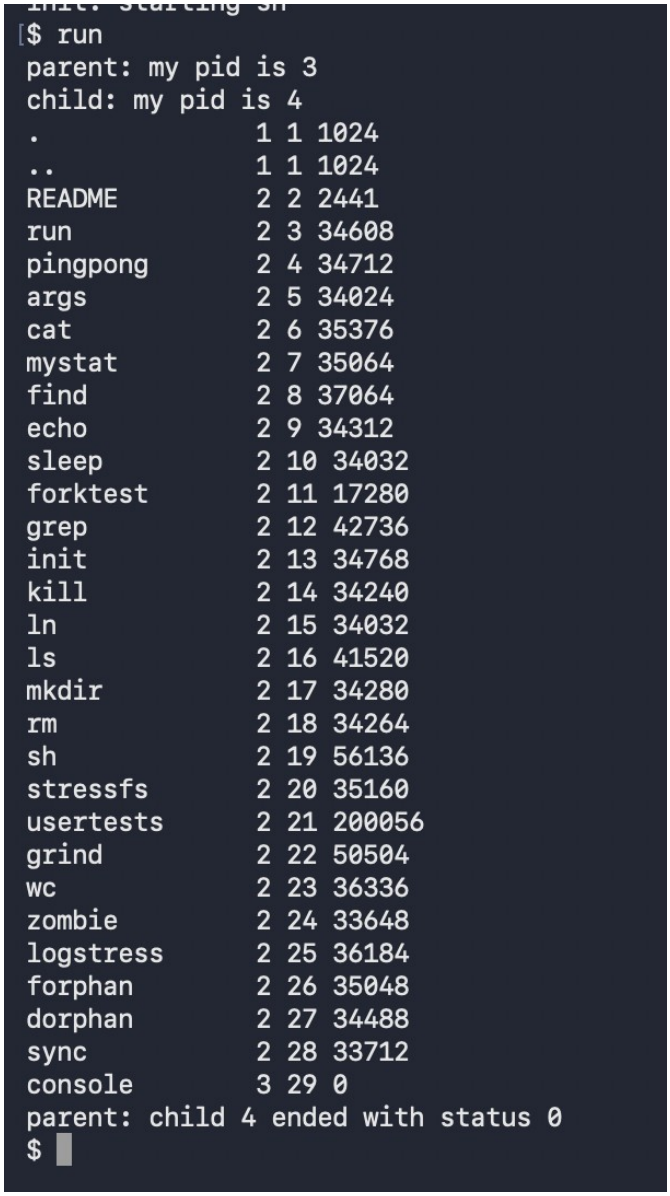


Figure 7 - xv6 run executing ls

2.4 Failed exec

The target was changed to nosuchprog. Because the executable did not exist, exec returned to the following line, which printed an error and called exit(1).

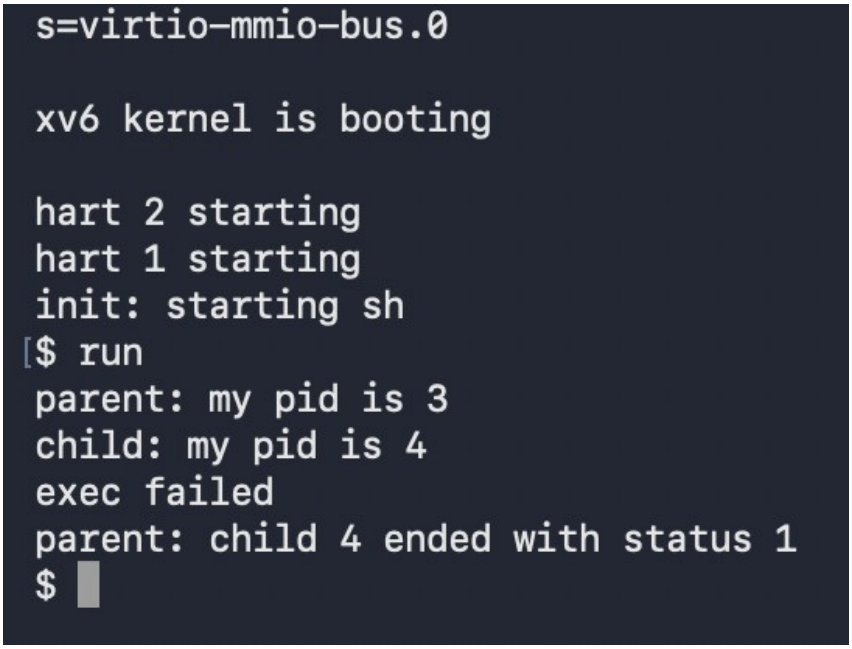


Figure 8 - xv6 failed exec and status 1

Observation

The failure status differs from Linux because the two programs explicitly selected different values: the xv6 source calls exit(1), while the Linux source calls exit(127).

2.5 Bidirectional Communication with pingpong

Two pipes were created because each pipe carried data in one direction. The parent wrote one byte to p2c, the child read it and printed the ping message, and the child returned the byte through c2p. The parent's read from c2p blocked until that reply was available.

```
run
pingpong
```

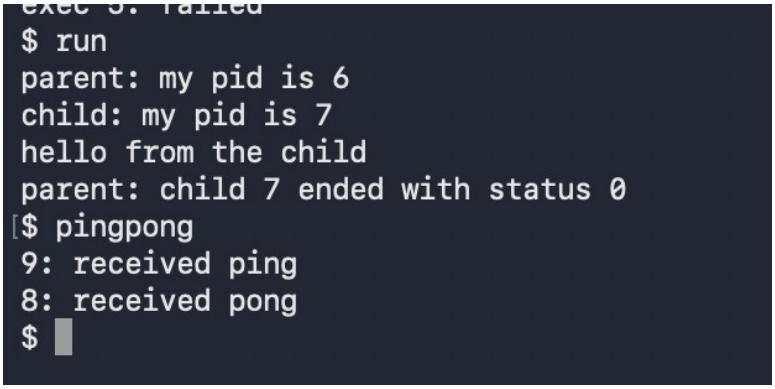


Figure 9 - Final xv6 verification of run and pingpong

Observation

Separate pipes prevent the two directions from competing for the same byte stream. The blocking read(c2p[0], &b, 1) makes the parent wait for the child's reply; wait(0) subsequently collects the terminated child. The observed order was ping first and pong second.

Appendix A Linux run.c

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void)
{
    printf("parent: my PID is %d\n", getpid());
    fflush(stdout);
    pid_t pid = fork();
    if (pid < 0) { perror("fork"); return 1; }
    if (pid == 0) {
        printf("child:  my PID is %d, my parent is %d\n", getpid(), getppid());
        fflush(stdout);
        execlp("ls", "ls", "-l", (char *)NULL);
        perror("exec");
        exit(127);
    }
    int status;
    waitpid(pid, &status, 0);
    printf("parent: child %d ended with status %d\n", pid, WEXITSTATUS(status));
    return 0;
}
```

Appendix B Linux zombie.c

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void)
{
    printf("parent: my PID is %d\n", getpid());
    fflush(stdout);
    pid_t pid = fork();
    if (pid < 0) { perror("fork"); return 1; }
    if (pid == 0) {
        printf("child:  my PID is %d, my parent is %d\n", getpid(), getppid());
        fflush(stdout);
        execlp("ls", "ls", "-l", (char *)NULL);
        perror("exec");
        exit(127);
    }
    int status;
    sleep(20);
    waitpid(pid, &status, 0);
    printf("parent: child %d ended with status %d\n", pid, WEXITSTATUS(status));
    return 0;
}
```

Appendix C Linux redirect.c

```
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void)
{
    pid_t pid = fork();
    if (pid == 0) {
        close(1);
        int fd = open("out.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);
        if (fd < 0) { perror("open"); exit(1); }
        fprintf(stderr, "open gave descriptor %d\n", fd);
        execlp("ls", "ls", "-l", (char *)NULL);
        perror("exec");
        exit(127);
    }
    waitpid(pid, NULL, 0);
    printf("done: the listing is in out.txt\n");
    return 0;
}
```

Appendix D Linux pipeline.c

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void)
{
    int p[2];
    if (pipe(p) < 0) { perror("pipe"); return 1; }
    if (fork() == 0) {
        close(1); dup(p[1]);
        close(p[0]); close(p[1]);
        execlp("ls", "ls", "/etc", (char *)NULL);
        perror("exec ls"); exit(127);
    }
    if (fork() == 0) {
        close(0); dup(p[0]);
        close(p[0]); close(p[1]);
        execlp("wc", "wc", "-l", (char *)NULL);
        perror("exec wc"); exit(127);
    }
    close(p[0]); close(p[1]);
    wait(NULL); wait(NULL);
    return 0;
}
```

Appendix E xv6 user run.c

```
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
  printf("parent: my pid is %d\n", getpid());
  int pid = fork();
  if(pid < 0){ printf("fork failed\n"); exit(1); }
  if(pid == 0){
    printf("child: my pid is %d\n", getpid());
    char *args[] = { "echo", "hello", "from", "the", "child", 0 };
    exec("echo", args);
    printf("exec failed\n");
    exit(1);
  }
  int status;
  wait(&status);
  printf("parent: child %d ended with status %d\n", pid, status);
  exit(0);
}
```

Appendix F xv6 user pingpong.c

```
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
  int p2c[2], c2p[2];
  char b;
  pipe(p2c); pipe(c2p);
  if(fork() == 0){
    close(p2c[1]); close(c2p[0]);
    read(p2c[0], &b, 1);
    printf("%d: received ping\n", getpid());
    write(c2p[1], &b, 1);
    close(p2c[0]); close(c2p[1]);
    exit(0);
  }
  close(p2c[0]); close(c2p[1]);
  b = 'x';
  write(p2c[1], &b, 1);
  read(c2p[0], &b, 1);
  printf("%d: received pong\n", getpid());
  close(p2c[1]); close(c2p[0]);
  wait(0);
  exit(0);
}
```

Conclusion and Critical Analysis

The two laboratory works demonstrated that shell behavior follows a small set of precise kernel rules. fork created a second execution context, exec replaced the child's program without creating another process, and wait or waitpid synchronized the parent while also removing the child's zombie record. The failed-exec experiments also showed that the exit status is chosen by the fallback code, not automatically standardized across the two environments.

The descriptor experiments provided the clearest connection between shell syntax and system calls. Closing descriptor 1 before open redirected an unchanged ls process because descriptors, rather than program-specific output settings, determine the destination. Similarly, the manual pipeline worked only when every process closed unused pipe ends. Retaining one write end prevented EOF and left wc blocked, illustrating why descriptor ownership matters across fork.

xv6 exposed the same principles with a much smaller interface. The run program reproduced the Linux fork-exec-wait lifecycle, while pingpong demonstrated blocking communication and the need for one pipe per direction. All required programs compiled and produced the expected results, and the combined experiments support one consistent model of processes, descriptors, synchronization, and interprocess communication.